Name: Chao Sivchhoeng

ID: e20240949

Group: I3-AMS-A

TP2

1. Python Basic Data Types

In Python, **everything is an object**, meaning that even basic data types act as reference types behind the scenes.

a. Primitive/Built-in Data Types (int, float, bool, str)

- Python officially calls these built-in types; "primitive" is a beginner-friendly grouping.

Type	Stand For	Meaning	Example
int	*Integer*	Represents whole numbers of arbitrary size, limited only by available system memory.	42, -7
float	*Floating-point*	Represents real numbers containing a decimal point.	3.14, -0.001
bool	*Boolean*	A subclass of int representing truth values: True or False. Internally, True evaluates to 1 and False evaluates to 0.	True / False
str	*String*	An **immutable** sequence of Unicode characters used to represent text.	"Hello", 'Python'

b. Collection and Reference Types (list, tuple, dict, set, objects)

- Collections store groups of items. They differ fundamentally by order, uniqueness, and mutability (whether they can be altered after creation).

Type	Ordered?	Mutable?	Unique Elements Only?	Syntax	Key Characteristic
list	Yes	Yes	No	[1, 2, 2]	Dynamic arrays that accept mixed types.
tuple	Yes	No	No	(1, 2, 2)	Memory-efficient; safe from unintended modification.
dict	Yes*	Yes	Keys must be unique	{"a": 1}	Key-value pairs optimized for O(1) lookups.

set	No	Yes	Yes	{1, 2, 3}	Eliminates duplicates; optimized for membership testing.

c. Literals in Python (numeric, string, Boolean, None, collection literals)

- Literals in Python are raw, fixed values written directly into the source code that represent constant data.
- **Numeric literals** represent numbers and are completely immutable. They are divided into three sub-types:
  - **Integer Literals:** Whole numbers without decimals. They can be written in decimal (base 10), binary (prefix 0b), octal (prefix 0o), or hexadecimal (prefix 0x).

```
x = 100       # Decimal
b = 0b1010    # Binary (equals 10)
```

  - **Floating-Point Literals:** Numbers with a decimal point or expressed in scientific notation.

```
pi = 3.14159  # Fractional
e = 2.5e3     # Exponential (2.5 * 10^3 = 2500.0)
```

  - **Complex Literals:** Numbers with a real and an imaginary part, written using j or J.

```
z = 3 + 4j
```

- **String literals** represent text data and are formed by wrapping characters in quotes.
  - **Single/Double Quotes:** Used for single-line strings:

```
msg1 = 'Hello'
msg2 = "Python"
```

  - **Triple Quotes (''' or """):** Used to write multi-line strings or docstrings.

```
multiline = """This spans
multiple lines."""
```

- **Boolean Literals** Represent truth values. Case sensitivity matters; only **True** and **False** are recognized.

```
is_active = True
has_passed = False
```

- **None literal:** Python features a single special literal called None to explicitly mark the absence of a value, an unassigned variable, or a null state.

```
empty_val = None
```

Practical example: storing student names, ages, scores, and student records.

```python
name = "Dara"
age = 20
score = 85.5
is_passed = True
remarks = None

subjects = ["Python", "Math", "English"]  #list
student_info = ("Dara", 20)               #tuple
scores = {"Python": 85.5, "Math": 90.0}   #dict
departments = {"Data Science", "IT"}      #set

#Student Record using a Dictionary
student = {
    "name": name,
    "age": age,
    "scores": scores,
    "is_passed": is_passed,
    "remarks": remarks,
    "subjects": subjects
}
#Display Student Information
print("Student Name:", student["name"])
print("Age:", student["age"])
print("Subjects:", student["subjects"])
print("Scores:", student["scores"])
print("Passed:", student["is_passed"])
print("Remarks:", student["remarks"])
print("Departments:", departments)
```

Output:

```
Student Name: Dara
Age: 20
Subjects: ['Python', 'Math', 'English']
Scores: {'Python': 85.5, 'Math': 90.0}
Passed: True
Remarks: None
Departments: {'Data Science', 'IT'}
```

2. Python Variables and Attributes

a. Local Variables

- **Local Variables** is a variable that is declared inside a function or block and can only be accessed within that function or block.

b. Instance Attributes

- **Instance Attributes** is a variable that belongs to a specific object (instance) of a class.

Example:

```
class student:
    def __init__(self ,name ,age):
        self.name =name
        self.age =age
student1 =student("data", 21)
student2 =student("daro", 25)
print (student1.age)
print(student2.name)
```

Output:

```
21
daro
```

c. **Class Attributes (shared by all instances)**

- **Class Attributes** is a variable defined inside a class but outside any method. It is shared by all instances of the class unless overridden by an instance attribute.

Example:

```
class dog:
    species = "Canis familiaris"

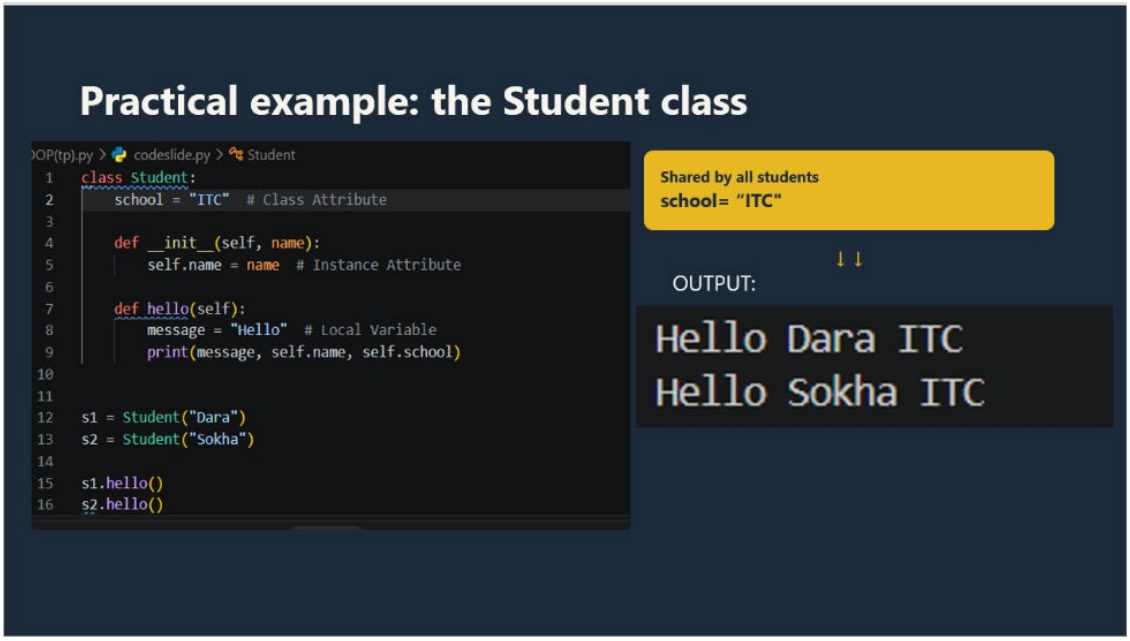
d1 = dog()
d2 =dog()

print(d1.species)
print(d2.species)
```

Output:

```
Canis familiaris
Canis familiaris
```

Practical example: a student class with individual names and a shared school name.



3. **Python Modifiers and Access Control – Part 1**
   a. **Public, Non-Public, and Private Naming Conventions**

- **Public members** are a public attribute and methods can be accessed from inside and outside a class.

Example:

```
# Public member
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

account = BankAccount("Dara", 1000)

print(account.owner)
print(account.balance)
```

Output:

```
Dara
1000
```

- **Non-public members (single underscore _)** is a single leading underscore attribute or method can be accessed for internal use only.

Example:

```python
# Non-public member
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self._balance = balance

account = BankAccount("Dara", 1000)

print(account.owner)
print(account._balance)
```

Output:

```
Dara
1000
```

- **Private-style members (double underscore__)** is a double leading underscore triggers a mechanism called mangling. It is hidden from outside access to prevent external code from accidentally modifying them.

Example:

```python
# Private-style members
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

account = BankAccount(1000)

#print(account.__balance)  # This will raise an AttributeError
# This will print the balance using name mangling
print(account._BankAccount__balance)
```

Output:

```
1000
```

b. <u>Access Control by Convention and Name Mangling</u>

- **Access control by convention** relies on naming rules to indicate which member should be accessed directly.
- **Name mangling** transforms double underscore attribute names to reduce accidental conflicts.

Example:

```python
# Access control by convention and name mangling
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self._account_type = "Savings"
        self.__balance = balance

account = BankAccount("Dara", 1000)

print(account.owner)
print(account._account_type)
print(account._BankAccount__balance)
```

Output:

```
Dara
Savings
1000
```

c. The @staticmethod Decorator

- **@staticmethod** is a function that lives inside the class but does not need an object or the class. This does not receive **self** or **cls**.

Example:

```python
# The @staticmethod decorator
class BankAccount:
    @staticmethod
    def is_valid_amount(amount):
        return amount > 0

print(BankAccount.is_valid_amount(500))
print(BankAccount.is_valid_amount(-100))
```

Output:

```
True
False
```

d. The @classmethod Decorator

- **@classmethod** is a class method receives the class as its first argument, which was named **cls**.

Example:

```python
# The @classmethod decorator
class BankAccount:
    bank_name = "ABA Bank"
    total_accounts = 0

    def __init__(self, owner):
        self.owner = owner
        BankAccount.total_accounts += 1

    @classmethod
    def show_bank_info(cls):
        print("Bank:", cls.bank_name)
        print("Total accounts:", cls.total_accounts)

account1 = BankAccount("Dara")
account2 = BankAccount("Sokha")

BankAccount.show_bank_info()
```

Output:

```
Bank: ABA Bank
Total accounts: 2
```

Practical Example: controlling access to a BankAccount class and defining class-level methods.

```python
# Practical Exercise
class BankAccount:
    bank_name = "ABC Bank"
    total_accounts = 0

    def __init__(self, owner, balance):
        self.owner = owner
        self._account_type = "Savings"
        self.__balance = balance
        BankAccount.total_accounts += 1

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Deposit successful")
        else:
            print("Invalid amount")

    def get_balance(self):
        return self.__balance
```

```python
    @staticmethod
    def is_valid_amount(amount):
        return amount > 0

    @classmethod
    def show_bank_info(cls):
        print("Bank:", cls.bank_name)
        print("Total accounts:", cls.total_accounts)

account = BankAccount("Dara", 1000)

print("Owner:", account.owner)
account.deposit(500)
print("Balance:", account.get_balance())

print("Valid amount:", BankAccount.is_valid_amount(200))
BankAccount.show_bank_info()
```

Output:

```
Owner: Dara
Deposit successful
Balance: 1500
Valid amount: True
Bank: ABC Bank
Total accounts: 1
```

4. **Python Modifiers and OOP Features – Part 2**

e. Abstract Classes and Methods (abc, @abstractmethod)

- **An abstract class** is a blueprint for other classes. It can define methods that subclasses must implement. Python uses **abc** and **@abstractmethod** to create abstract classes and methods.

Example:

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def area(self):
        return 3.14 * 5 * 5

c = Circle()
print(c.area())
```

Output:

```
78.5
```

f. Access Control and Encapsulation

- **Encapsulation** means keeping data and methods together inside a class and controlling access to the data.

Python uses naming conventions to indicate three levels of access:

- **Public:** name —>can be accessed from outside the class.
- **Protected:** _name —> intended for use inside the class and its subclasses.
- **Private:** __name —> makes direct access more difficult through name mangling.

Example:

```
class Student:
    def __init__(self):
        self.name = "jungkook"
        self.__score = 90

    def show_score(self):
        print(self.__score)

s = Student()

print(s.name)
s.show_score()
```

Output:

```
jungkook
90
```

g. Other OOP Features: @property, @final, and typing.final

- The **@property decorator** allows us to access methods like an attribute. It is useful when we want to read a value without calling a method using parentheses.

Example:

```
class Student:
    def __init__(self):
        self._score = 90

    @property
    def score(self):
        return self._score

s = Student()
print(s.score)
```

Output:

```
90
```

- The **@final decorator** indicates that a method should not be overridden by a subclass. It helps programmers maintain the intended design of a class. However, Python does not enforce this restriction at runtime.

Example:

```
from typing import final

class Animal:
    @final
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

d = Dog()
d.sound()
```

Output:

```
Bark
```

- The **typing.final** is the decorator imported from Python's typing module. It can mark a class as one that should not be inherited from, or a method as one that should not be overridden.

Example:

```
from typing import final

@final
class Calculator:
    def add(self, a, b):
        return a + b

c = Calculator()
print(c.add(9, 10))
```

Output:

```
19
```

Practical example: an abstract Shape class with Circle and Rectangle subclasses.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Circle(Shape):
    def area(self):
        return 3.14 * 5 * 5
class Rectangle(Shape):
    def area(self):
        return 4 * 6
c = Circle()
r = Rectangle()

print("Circle area:", c.area())
print("Rectangle area:", r.area())
```

Output:

```
Circle area: 78.5
Rectangle area: 24
```

5. **Python Operators – Part 1**

   a. **Arithmetic Operators (+, -, *, /, //, %, **)**

- **Arithmetic operators** are used to perform mathematical calculations such as addition, subtraction, multiplication, and division.

Example:

```
# a. Arithmetic
a = 10
b = 3

print("Addition:", a + b)
print("Substraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponentiation power:", a ** b)
```

Output:

```
Addition: 13
Substraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulus: 1
Exponentiation power: 1000
```

   b. **Comparison Operators (==, !=, >, <, >=, <=)**

- **Comparison operators** are used to compare two values and return either **True** or **False**.

Example:

```
# b. Comparison
score = 85

print("Equal:", score == 85)
print("Not Equal:", score != 85)
print("Greater Than:", score > 85)
print("Less Than:", score < 85)
print("Greater Than or Equal:", score >= 85)
print("Less Than or Equal:", score <= 85)
```

Output:

```
Equal: True
Not Equal: False
Greater Than: False
Less Than: False
Greater Than or Equal: True
Less Than or Equal: True
```

c. <u>Bitwise Operators (&, |, ^, ~, <<, >>)</u>

- **Bitwise operators** are used to perform operations on the binary bits of integer.

Example:

```
a=10
b=3
print("Bitwise Operators (&, |, ^, ~, <<, >>)")
# botwisw and
print(a&b)
# bitwise or
print(a|b)
#bitwise XOR
print(a^b)
#bitwise NOt for intergers
print(~3)
#bitwise shitf
print(a<<b)
print(a>>b)
```

Output:

```
Bitwise Operators (&, |, ^, ~, <<, >>)
2
11
9
-4
80
1
```

d. Logical Operators (and, or, not)

- **Logical operators** are used to combine or reverse conditions and return Boolean values (**True** or **False**).

Example:

```
# d. Logical
score = 85
attendance = 90

print("Logical AND:", score > 80 and attendance > 85)
print("Logical OR:", score > 80 or attendance < 85)
print("Logical NOT:", not(score > 80))
```

Output:

```
Logical AND: True
Logical OR: True
Logical NOT: False
```

Practical example: calculating student grades and checking eligibility.

```
math = 18
physics= 35
english =42
attendance =90
total_score = math + physics +english
average = total_score/3
passed =average>=50
eligible = average>=80 and attendance>=75

print("total_score =",total_score)
print("Average:", average)
if average >= 90:
    print("Grade: A")
elif average >= 80:
    print("Grade: B")
elif average >= 70:
    print("Grade: C")
elif average >= 60:
    print("Grade: D")
elif average >= 50:
    print("Grade: E")
else:
    print("Grade: F")
print("Exam result:", "PASS" if passed else "FAIL")
print("Award eligibility:", "ELIGIBLE" if eligible else "NOT ELIGIBLE")
```

Output:

```
total_score = 95
Average: 31.666666666666668
Grade: F
Exam result: FAIL
Award eligibility: NOT ELIGIBLE
```

6. **Python Operators – Part 2**
   e. **Assignment Operators (=, +=, -=, *=, etc.)**

- The **assignment operator** is the = symbol. It binds a variable (name) to a value or object in memory. Python also supports **augmented assignment operators** like +=, -=, *=, /=, and the **walrus operator** :=, Other assignment operators include (//=, %=, **=, &=, |=, ^=, <<=, and >>=).

Example:

```
a = 10    # Assigns integer 10 to variable a
print(a)
a += 3    # Equivalent to a = a + 3 → a becomes 13
print(a)
a -= 2    # Equivalent to a = a - 2 → a becomes 11
print(a)
a *= 4    # Equivalent to a = a * 4 → a becomes 44
print(a)
a /= 2    # Equivalent to a = a / 2 → a becomes 22.0
print(a)
```

Output:

```
10
13
11
44
22.0
```

f. <u>Special Operators: Identity (is, is not), Membership (in, not in), and Conditional Expressions (x if condition else y)</u>

- The **identity operators** are used to check whether two variables refer to the **same object in memory**, not just whether their values are equal. This is different from the equality operator (==), which compares the values of objects.

Python provides **two identity operators**:

- **is** → Returns True if both variables point to the same object (same memory location).
- **is not** → Returns True if both variables point to different objects.

Example:

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a is c)
print(a is b)
print(a is not b)
```

Output:

```
True
False
True
```

- **Membership operators** are used to check whether a value exists **inside** a sequence (like a list, tuple, string, set, or dictionary).

Types:

- **in** → Returns True if the value is found in the sequence.
- **not in** → Returns True if the value is **not** found in the sequence.

Example:

```
fruits = ["apple", "banana", "cherry"]

print("banana" in fruits)
print("orange" not in fruits)
```

Output:

```
True
True
```

- A **conditional expression** selects one of two values depending on whether a condition is true or false. It is a short way to write a simple if-else statement.

Example:

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

Output:

```
Adult
```

g. Operator Precedence and Associativity

- **Operator precedence** determines which operator is evaluated first in an expression. Associativity determines the evaluation order when operators have the same precedence.

Example:

```
a = 2 + 3 * 4
b = (2 + 3) * 4
c = 10 - 3 - 2
print(a)
print(b)
print(c)
```

Output:

```
14
20
5
```

Practical example: updating object attributes, checking membership in collections, and evaluating expressions.

```
class Student:
    def __init__(self, score):
        self.score = score
s = Student(60)

s.score += 10

subjects = ["Python", "Math", "Physics"]

print("Updated score:", s.score)
print("Study Python:", "Python" in subjects)

result = "Pass" if s.score >= 50 else "Fail"
print("Result:", result)

print("Expression:", 2 + 3 * 4)
```

Output:

```
Updated score: 70
Study Python: True
Result: Pass
Expression: 14
```

7. **Python Decision-Making Statements – Part 1**
   a. **The if Statement**

- **if statement** is used executes a block of code only when a specified condition is true.

Example:

```
# The if statement
score = 85

if score >= 50:
    print("Student passed.")
```

Output:

```
Student passed.
```

b. The if-else Statement

- **if-else statement** is used executes one block of code if the condition is true and another block if the condition is false.

Example:

```
# The if-else statement
score = 40

if score >= 50:
    print("Student passed.")
else:
    print("Student failed.")
```

Output:

```
Student failed.
```

c. The if-elif-else Statement

- **if-elif-else statement** checks multiple conditions in order and executes the first true block. If none of the conditions are true, runs **else** block.

Example:

```
#The if-elif-else statement
score = 85

if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
elif score >= 70:
    print("Grade C")
elif score >= 60:
    print("Grade D")
elif score >= 50:
    print("Grade E")
else:
    print("Grade F")
```

Output:

```
Grade B
```

Practical Example: determining student's pass/fail status and assigning letter grades.

```python
#Practical Example
score = float(input("Enter student score (0-100): "))

if score < 0 or score > 100:
    print("Invalid score. Please enter a score between 0 and 100.")
elif score >= 90:
    print("Result: Pass")
    print("Grade: A")
elif score >= 80:
    print("Result: Pass")
    print("Grade: B")
elif score >= 70:
    print("Result: Pass")
    print("Grade: C")
elif score >= 60:
    print("Result: Pass")
    print("Grade: D")
elif score >= 50:
    print("Result: Pass")
    print("Grade: E")
else:
    print("Result: Fail")
    print("Grade: F")
```

Output:

```
Enter student score (0-100): 95
Result: Pass
Grade: A
Enter student score (0-100): 85
Result: Pass
Grade: B
Enter student score (0-100): 75
Result: Pass
Grade: C
Enter student score (0-100): 65
Result: Pass
Grade: D
Enter student score (0-100): 55
Result: Pass
Grade: E
Enter student score (0-100): 45
Result: Fail
Grade: F
Enter student score (0-100): 110
Invalid score. Please enter a score between 0 and 100.
```

8. Python Decision-Making – Part 2

d. Nested if Statements

- Nested **if** Statements are an if statement placed inside another if statement. It is used to check another condition only when the first condition is true.

Example:

```
score = 95
if score >=50:
    print("pass")
    if score >=80:
        print("grande A")
```

✓ 0.0s

```
pass
grande A
```

e. <u>Multiple-Branch Selection Using match-case (Python 3.10+) or if-elif-else</u>

- Multiple-Branch Selection Using **match-case (Python 3.10+)** is a control structure that allows a program to choose one action from several alternatives based on different conditions or values.

Example:

```
day =2
match day:
    case 1:
        print("monday")
    case 2:
        print("tuesday")
    case 3:
        print("wendesday")
    case _:
        print("invalid day")
```

✓ 0.0s

```
tuesday
```

- if-elif-else is used to check multiple conditions, the first condition that is true will be executed.

Example:

```
score = 80
if score >=90:
    print("grande A")
elif score >=80:
    print("grande B")
elif score >=50:
    print("pass")
else:
    print("fail")
```

✓ 0.0s

```
grande B
```

f. <u>Conditional Expressions (condition ? value1 : value2 in Java becomes value1 if condition else value2 in Python)</u>

- **Conditional Expression** is an expression that returns one of two values depending on whether a condition.

Example:

```
score =40
result = "pass" if score>=50 else "fail"
print(result)
```

✓ 0.0s

```
fail
```

Practical example: a menu-driven student management system.

```
=========Student Mangement System===============
1. Add Student
2. View students
3. Exit
=========Student Mangement System===============
1. Add Student
2. View students
3. Exit
student : ['panda']
=========Student Mangement System===============
1. Add Student
2. View students
3. Exit
goodbye!
```

```python
student =[]
while True:
    print("=========Student Mangement System===============")
    print("1. Add Student")
    print("2. View students")
    print("3. Exit")

    choice = input("Enter your choice :")
    if choice == "1":
        name = input("Enter student name:")
        student.append(name)
    elif choice =="2":
        print("student :" , student)
    elif choice =="3":
        print("goodbye!")
        break

    else:
        print("invalid choice!")
```

✓ 31.6s

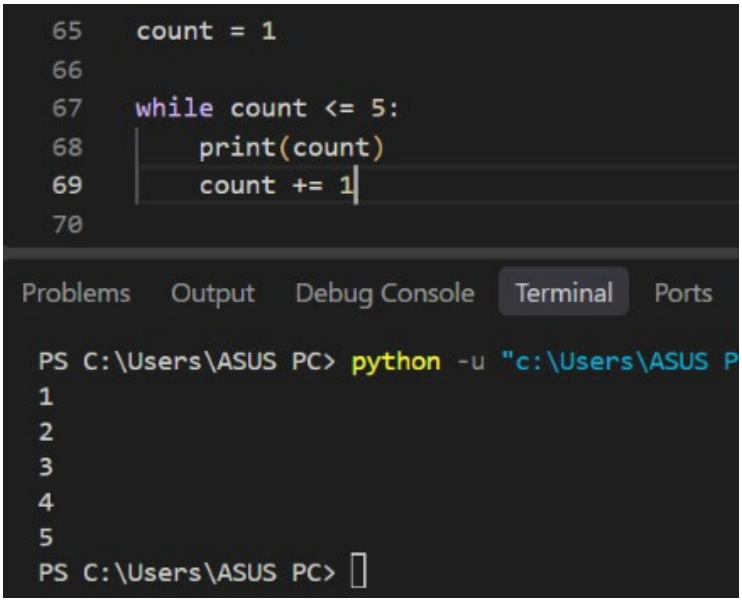
9. Python Loops and Loop Control

A **loop** is a programming structure that repeats a block of code multiple times. Instead of writing the same instructions repeatedly, we can use a loop to execute them efficiently.

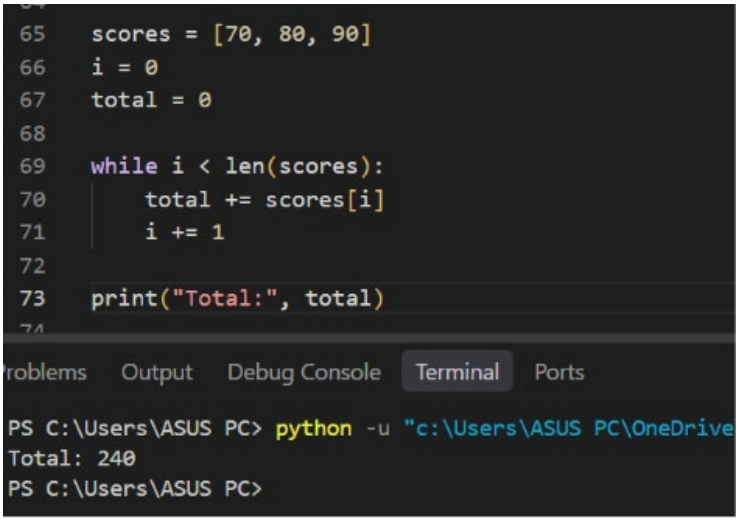
a. The while Loop

- A **while loop** repeats a block of code as long as its condition is True. It is especially useful when we do not know in advance how many repetitions will be required.

Example 1: Printing numbers



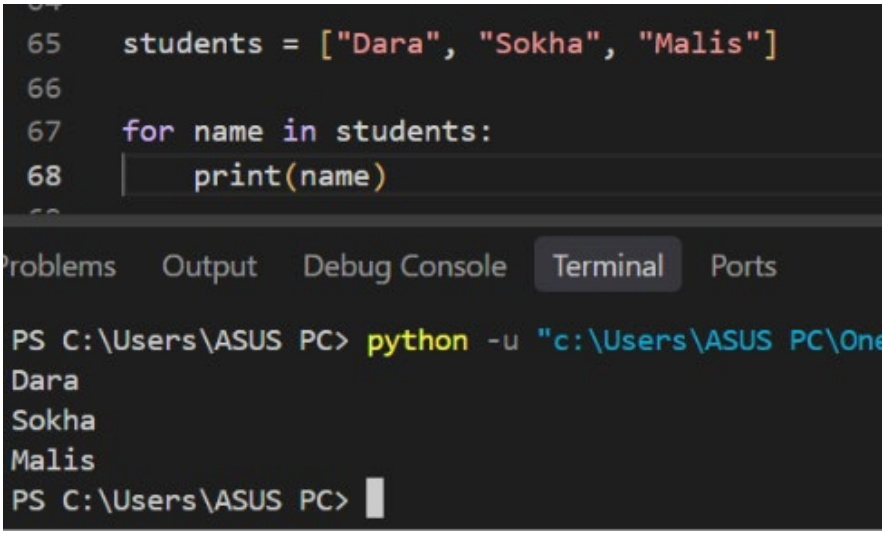
Example 2: Use a while loop to calculate the total score of a student.



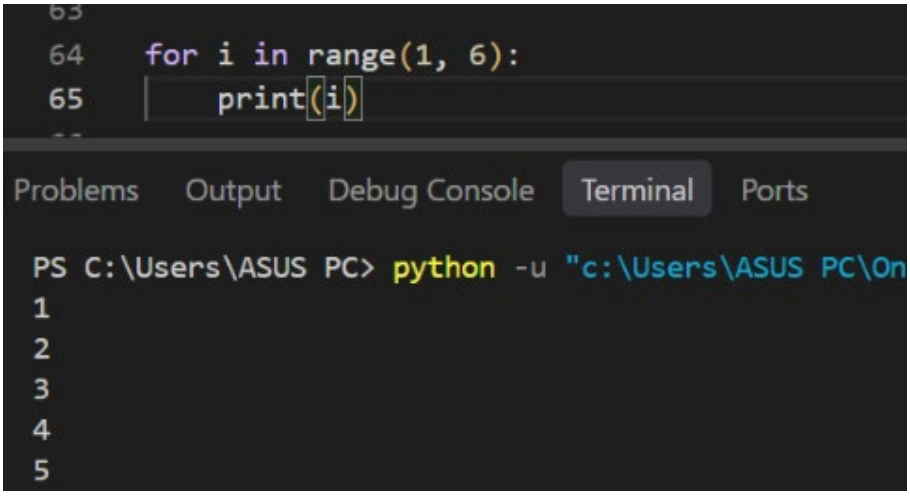
b. <u>The for Loop</u>

- A **for loop** repeats a block of code by taking items one at a time from an iterable, such as a list, tuple, string, dictionary, or range().

Example 1: Printing student names



Example 2: The range() function generates a sequence of integers, commonly used to control the number of iterations.



Example 3: Calculating the sum of scores

```
65   scores = [70, 80, 90]
66   total = 0
67
68   for score in scores:
69       total += score
70
71   print("Total:", total)
72
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\ASUS PC> python -u "c:\Users\ASUS PC\
Total: 240
PS C:\Users\ASUS PC>
```

c. Repetition with for and while (Python has no native do-while loop)

- Both loops repeat instructions, but they are best suited to different situations.
- Python does not have a native do-while loop. In some programming languages, a do-while loop executes the body first and checks the condition afterward. Therefore, its body runs at least once.
- In Python, we can achieve similar behavior by combining while True with break.

Example:

```
65   while True:
66       answer = input("Enter yes to continue: ")
67
68       if answer == "yes":
69           break
70
71       print("Please try again.")
72
73   print("Finished!")
74
75
76
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\ASUS PC> python -u "c:\Users\ASUS PC\OneDrive\Documents\YEAR3\
Enter yes to continue: No
Please try again.
Enter yes to continue: yes
Finished!
```

Explanation:

1. *while* True starts a loop that would continue indefinitely.
2. The program requests input from the user.
3. If the answer is "yes", *break* terminates the loop.
4. Otherwise, the program displays a message and asks again.

- This structure is useful when an action must happen at least once before the program decides whether to repeat it.

d. <u>Loop Control Statements: break, continue, and pass</u>

- **break:** Stop the loop. Immediately exits the nearest enclosing loop when the statement executes.

Example 1: Find the first number greater than 10

```
63
64  numbers = [4, 7, 9, 12, 15]
65
66  for number in numbers:
67      if number > 10:
68          print("Found:", number)
69          break
70
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\ASUS PC> python -u "c:\Users\ASUS PC\One
Found: 12
```

- (The loop checks 4, 7, 9, and 12. When it reaches 12, the condition is true and break terminates the loop. The number 15 is never checked.)[Text Wrapping Break]
- **continue:** Skip the current iteration. Skips the remaining statements in the current iteration and proceeds to the next one.

Example 2:

```
63
64    scores = [35, 72, 48, 90]
65
66    for score in scores:
67        if score < 50:
68            continue
69
70        print(score)
71
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\ASUS PC> python -u "c:\Users\ASUS PC\C
72
90
```

- (When the score is below 50, continue skips the print() statement and moves to the next score. Therefore, only scores of 50 or above are displayed.)
- **pass**: Do nothing. Acts as a placeholder when Python requires a statement, but no action needs to be performed yet.

Example 3:

```
64    scores = [35, 72, 48, 90]
65
66    for score in scores:
67        if score < 50:
68            pass
69
70        print(score)
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\ASUS PC> python -u "c:\Users\ASUS PC\OneDrive\
35
72
48
90
```

- (Unlike continue, pass does not skip the print() statement. All scores are displayed because pass simply does nothing.)

Practical example: iterating through student objects, calculating averages, and searching for a student.

```
#student objects
students = [
    Student("Dara", [80, 90, 85]),
    Student("Sokha", [10, 15, 32]),
    Student("Lina", [90, 95, 88])
]
#display students and calculate averages
lowest_average = min(student.average() for student in students)
#display students except the one with the lowest average
for student in students:
    if student.average() == lowest_average:
        continue  # Skip the student with the lowest average

    print(student.name, "Average:", student.average())
#search for a student
search_name = input("Enter student name to search: ")
i = 0

while i < len(students):
    if students[i].name.lower() == search_name.lower():
        print("Found:", students[i].name, 'with average score of', students[i].average())

        break
    i += 1
else:
    print("Student not found")

#Pass statement
for student in students:
    if student.name == "Dara":
        pass  # Skip Dara
    print(student.name)
```

Output:

```
Dara Average: 85.0
Lina Average: 91.0
Enter student name to search: Dara
Found: Dara with average score of 85.0
Dara
Sokha
Lina
```